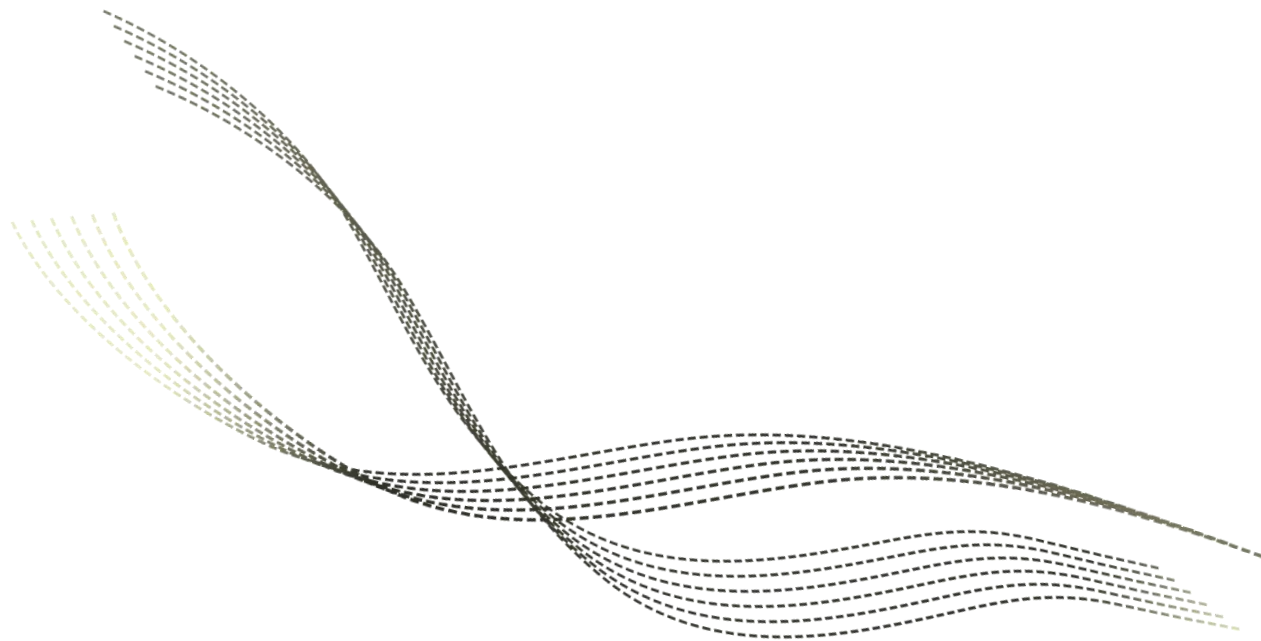


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路布线方法

沈明华



目录

CONTENTS

01

布线

02

通用布线算法

03

全局布线算法

04

详细布线算法

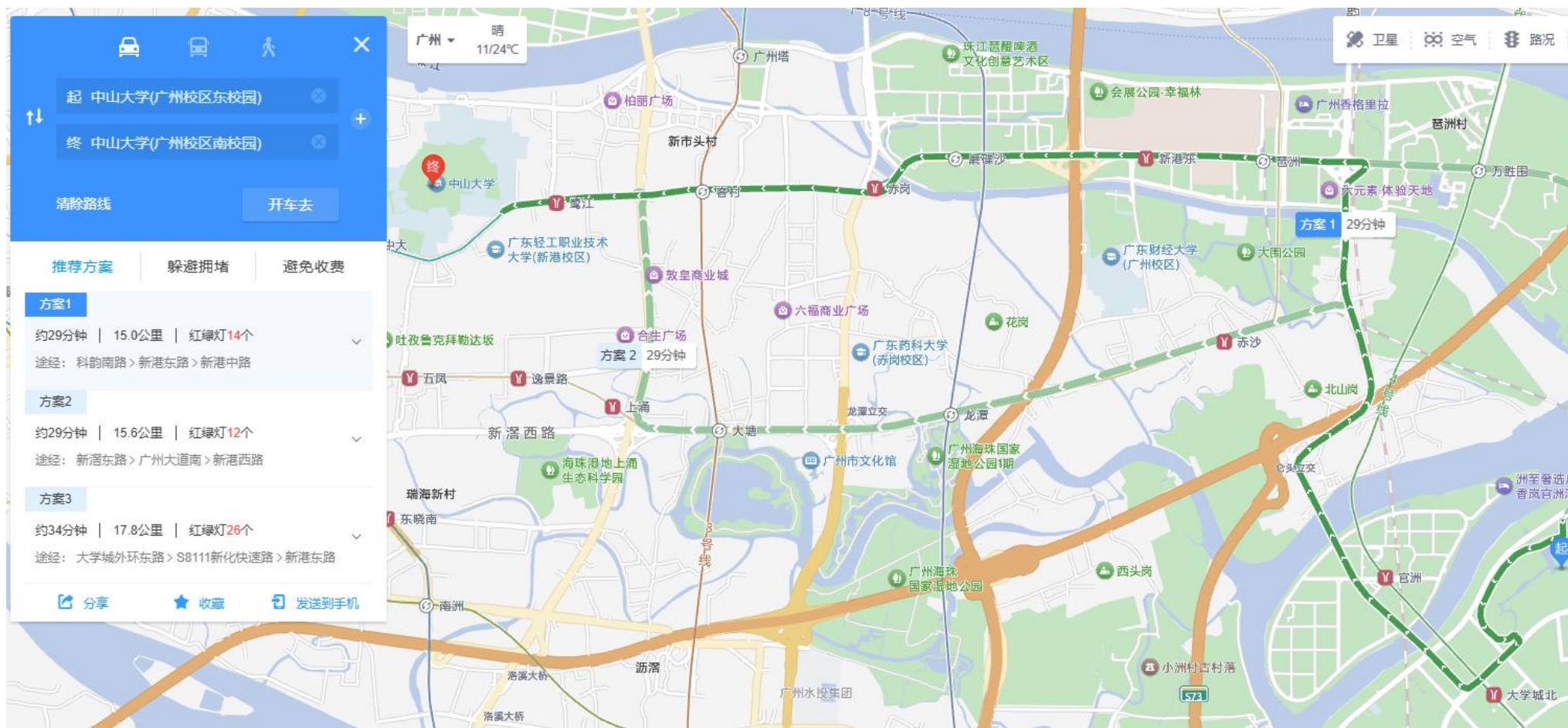
PART 01

布线

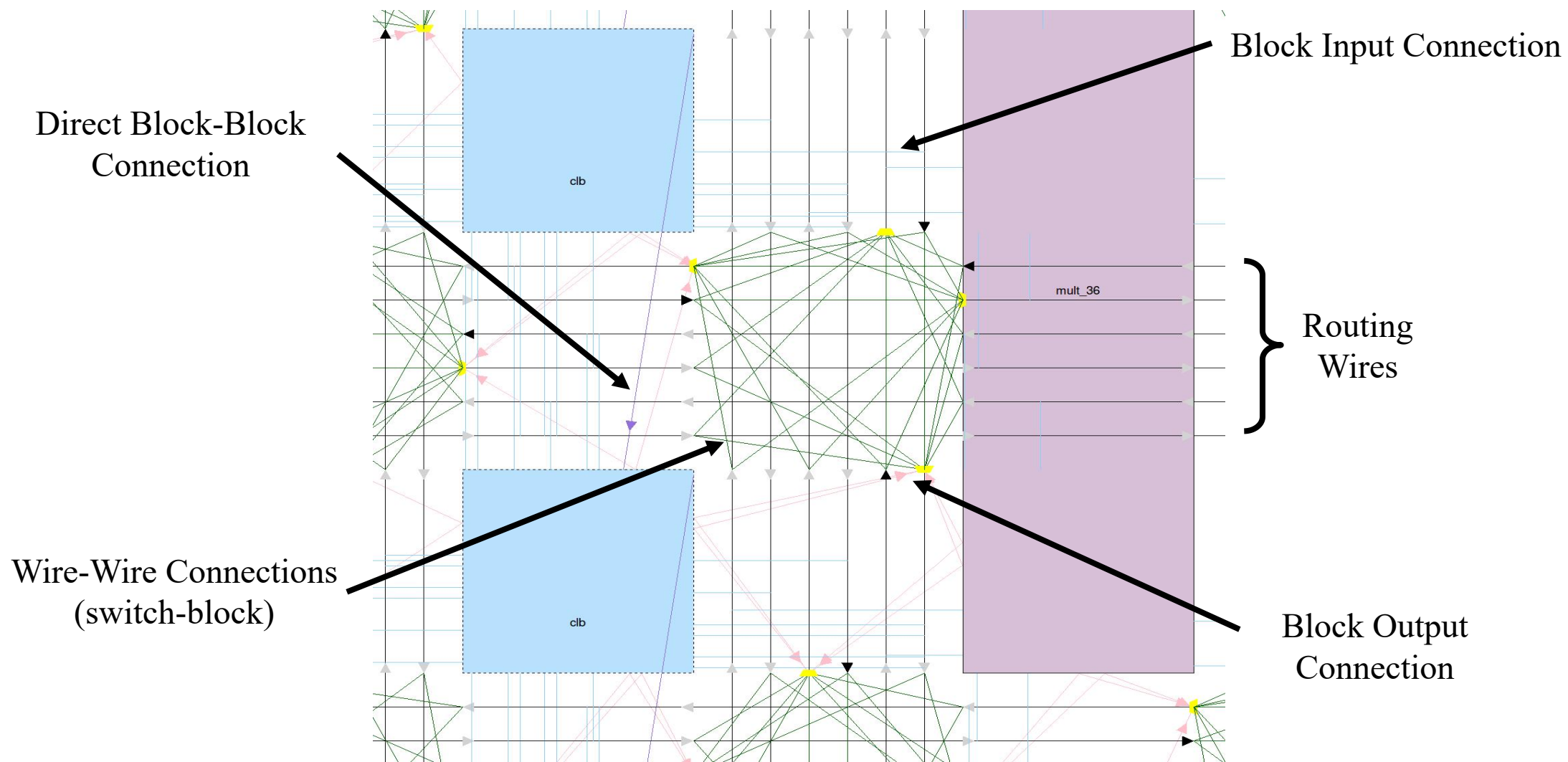
■ 布线

□ 以东校园前往南校园为例

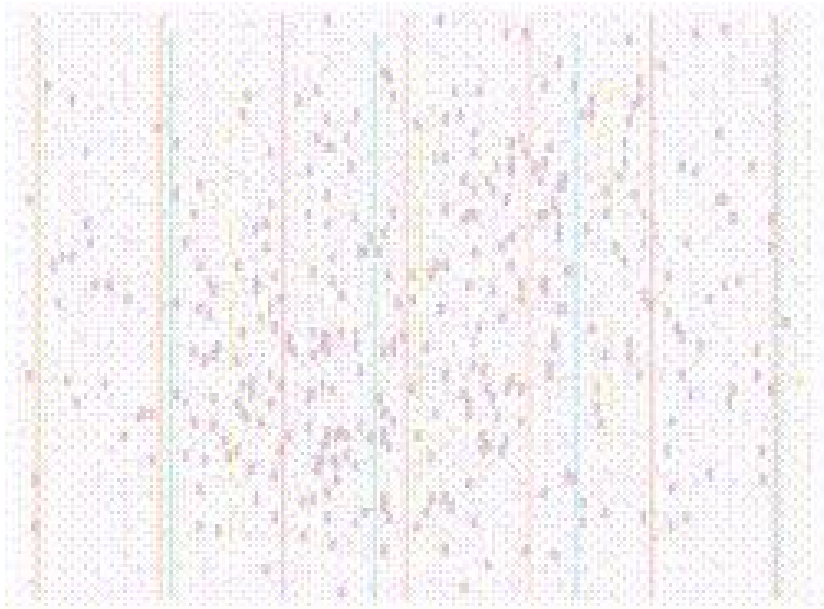
- 设计路线，使得路程时间最短，**并且**路途尽可能舒适。



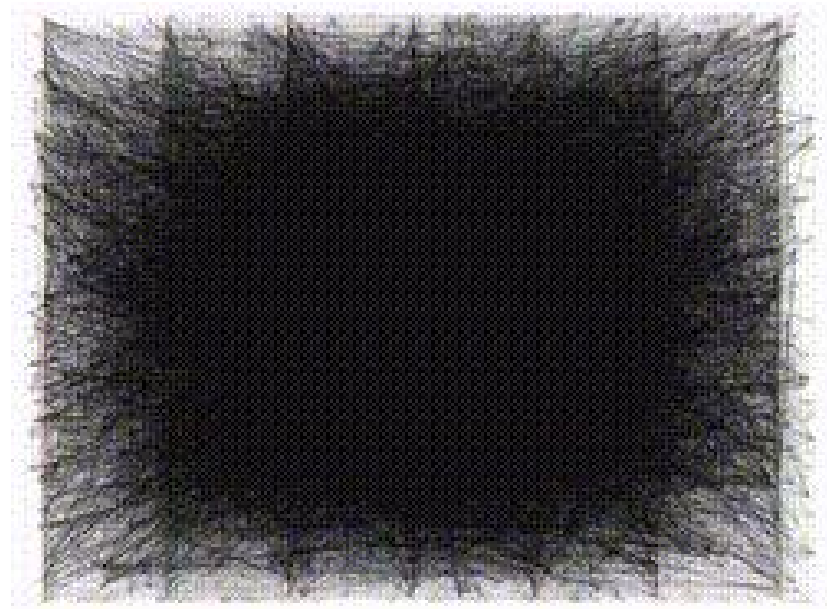
■ 布线结构



■ 布线示例



布局



布线

■ 布线问题

□ 布线问题定义

– 输入

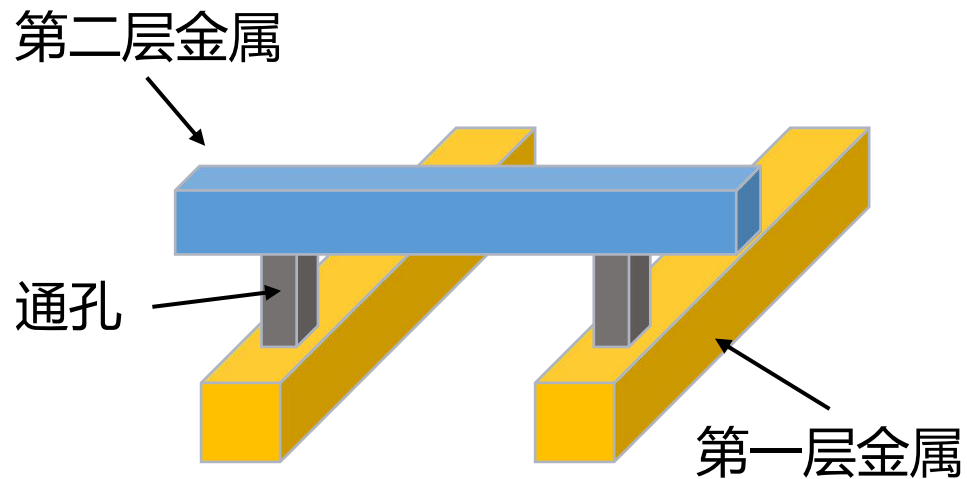
- 网表
- 针对关键网络的时序约束
- 块的位置和引脚的位置

– 输出

- 所有线网的几何布局

– 目的

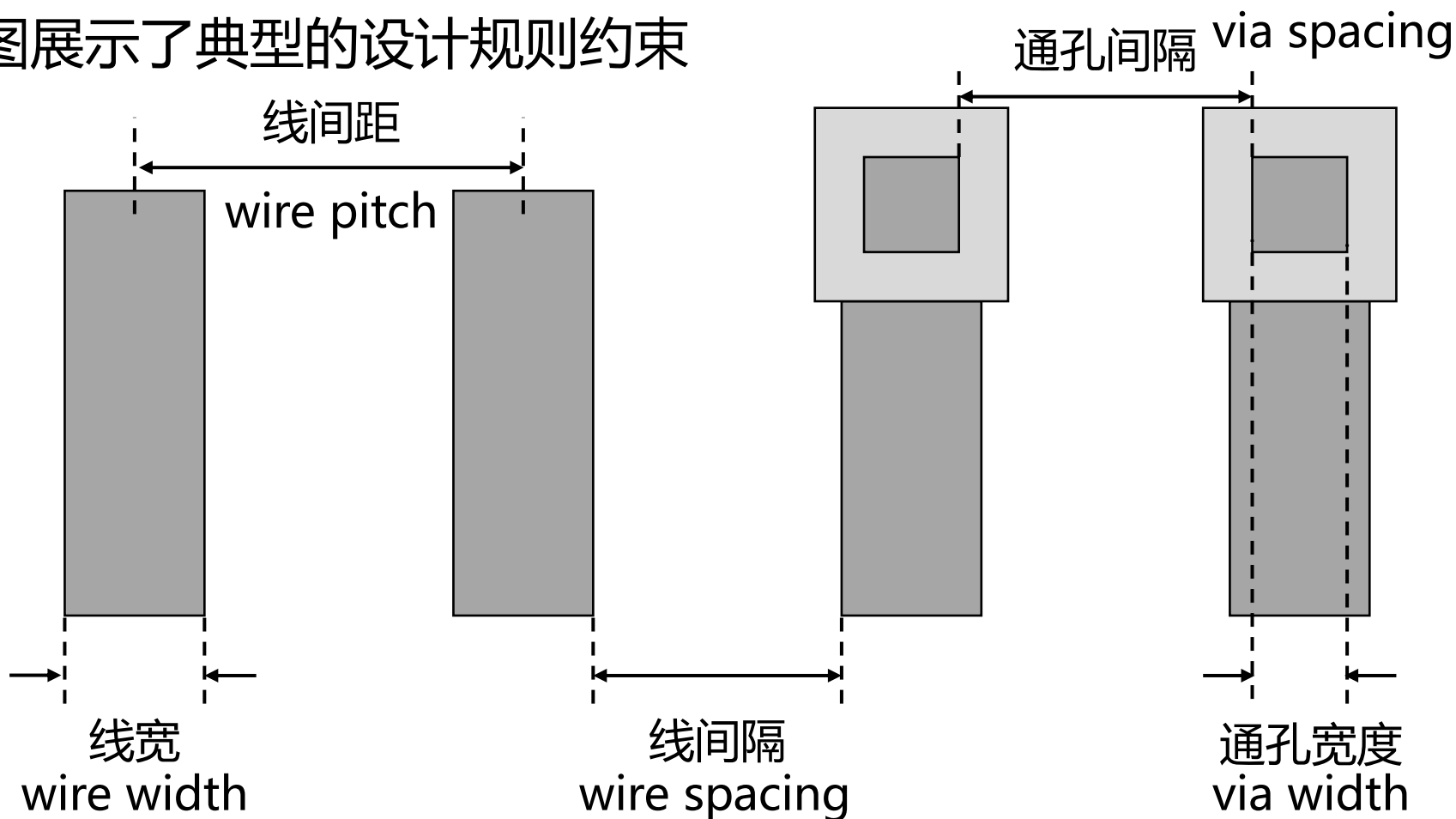
- 最小化总布线长度、通孔数量，或仅完成所有连接而不增加芯片面积
- 确保每个网络满足其时序约束



■ 布线约束

□ 布线约束

- 分为两个大类：设计规则(Design Rule Check)约束和性能约束
- 下图展示了典型的设计规则约束



■ 布线约束

□ 布线约束

– 设计规则约束

- 除了上述提到的设计规则约束之外，还包括制造工艺相关的其他规则，例如各金属层的电阻电容参数等

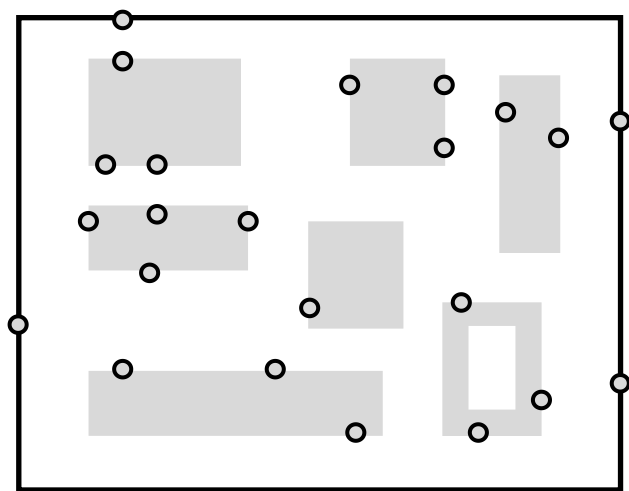
– 性能约束

- 确保连接满足芯片设计者提供的性能指标
- 例如，对于高速芯片而言，时序约束是最重要的性能约束

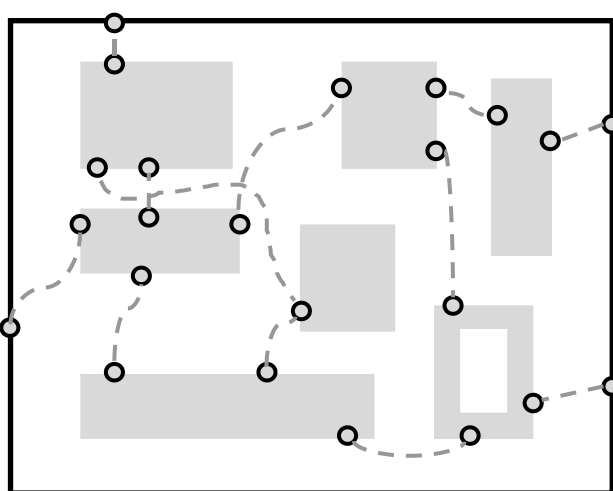
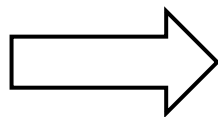
■ 布线过程

□ 布线问题通常用两阶段方法解决——全局布线和详细布线

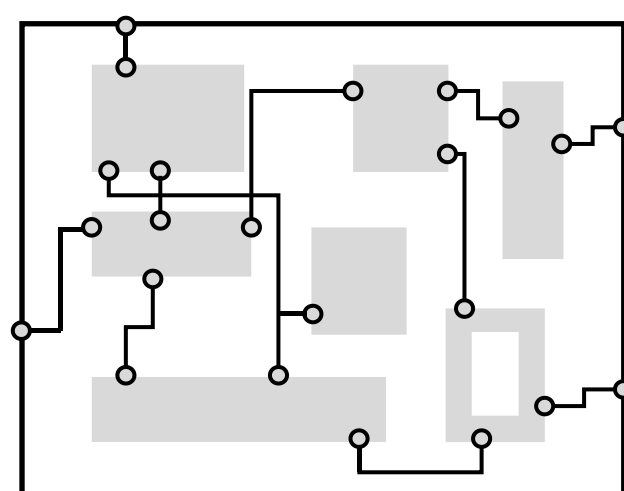
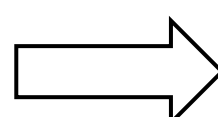
- 全局布线首先将布线区域划分为若干个单元块，并为所有线网确定单元块之间的路径
- 详细布线会为线网分配实际的走线和通孔位置
- 下图展示了全局布线与详细布线的流程



(a) 一个布局结果



(b) 全局布线



(c) 详细布线

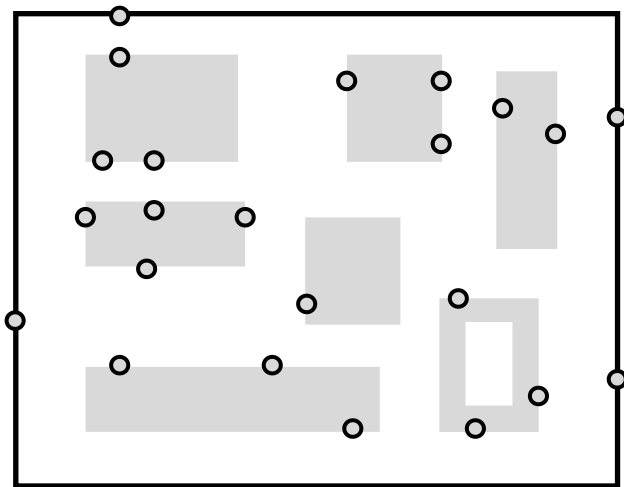
■ 布线过程

□ 图(b)展示全局布线的结果

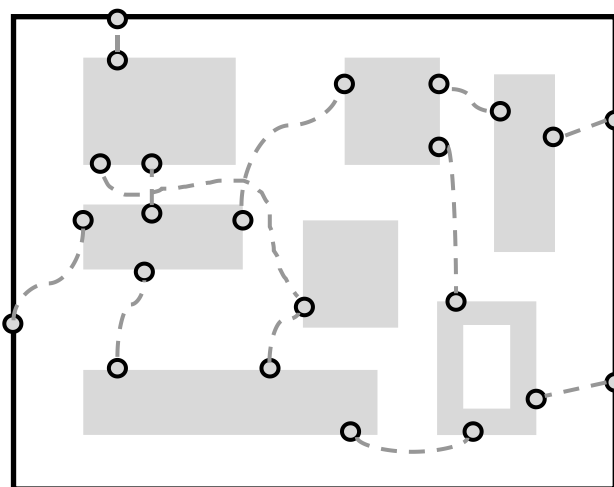
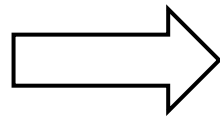
- 首先将布线区域划分为单元块，随后通过寻找连接引脚的单元块路径，为每个连接生成"宽松"的走线方案

□ 图(c)展示详细布线的结果

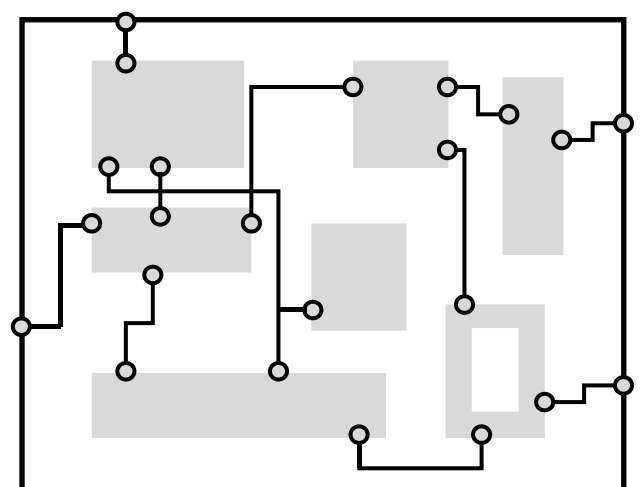
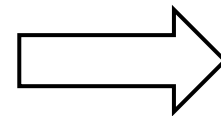
- 该阶段通过在单元块路径内进行搜索，最终确定每个网络的物理路径



(a)一个布局结果



(b)全局布线



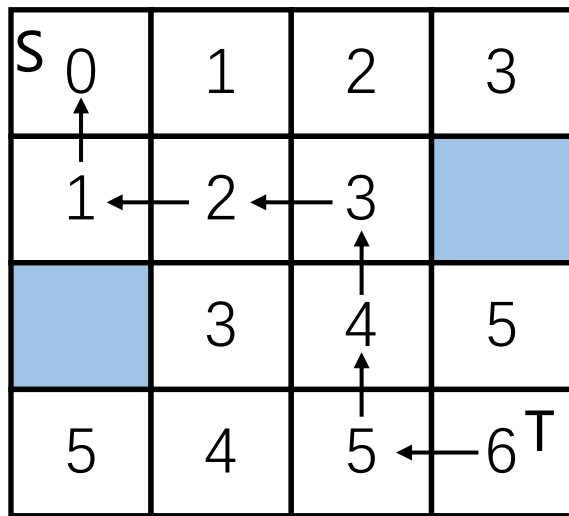
(c)详细布线

PART 02

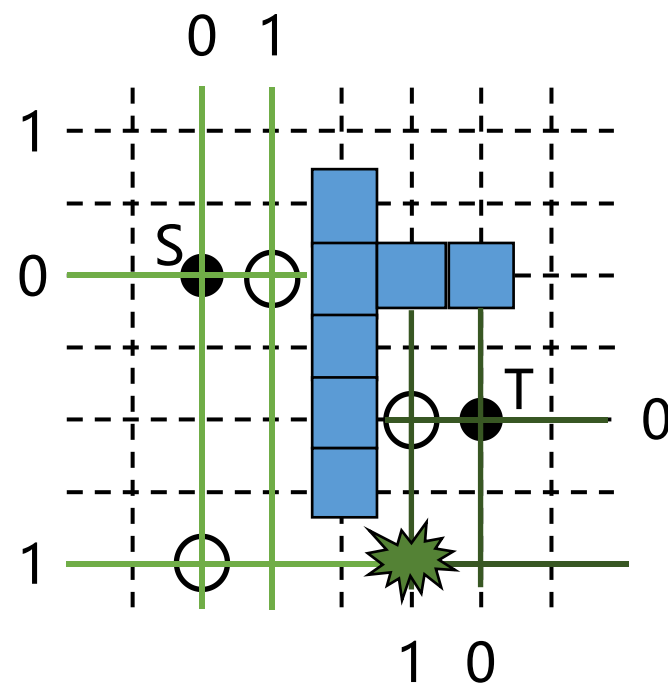
通用布线算法

■ 通用布线算法

- 通用布线算法可以应用于全局布线和详细布线上
- 下面介绍两类通用布线算法
 - 迷宫布线算法 (Maze routing algorithm)
 - 线搜索算法 (Line search algorithm)



迷宫布线



线搜索

■ 通用布线算法

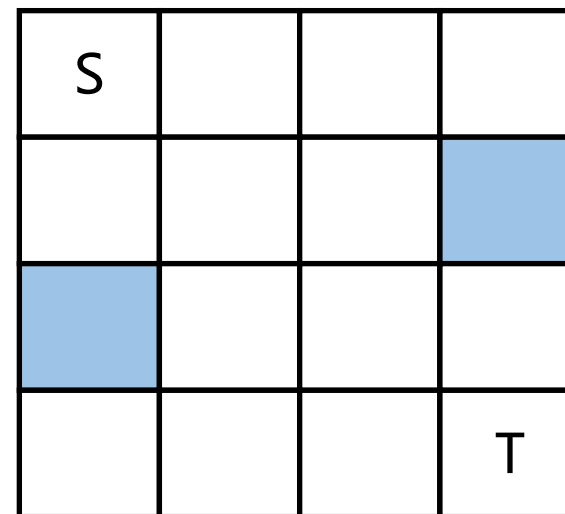
□ 迷宫布线算法 (Maze routing algorithm)

– 输入

- 一个平面矩形网格图
- 图上的两个点S和T
- 障碍物建模为阻塞的顶点

– 目标

- 找到连接S和T的最短路径



■ 通用布线算法

□ 迷宫布线算法也称李氏算法 (Lee's Algorithm)

- 在网格图中进行广度优先搜索
- 包含两个阶段
 - 波传播
 - 回溯

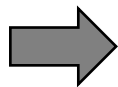
S ₀	1	2	3
1	2	3	
	3	4	5
5	4	5	6 ^T

■ 通用布线算法

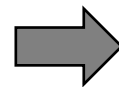
□ 波传播

- 从源节点S出发，根据波前与S的距离值逐格标记相邻网格单元，直至抵达目标节点T。
- 波前与S的距离为曼哈顿距离

S0			
			T



S0	1	2	3
1	2	3	
	3		
			T

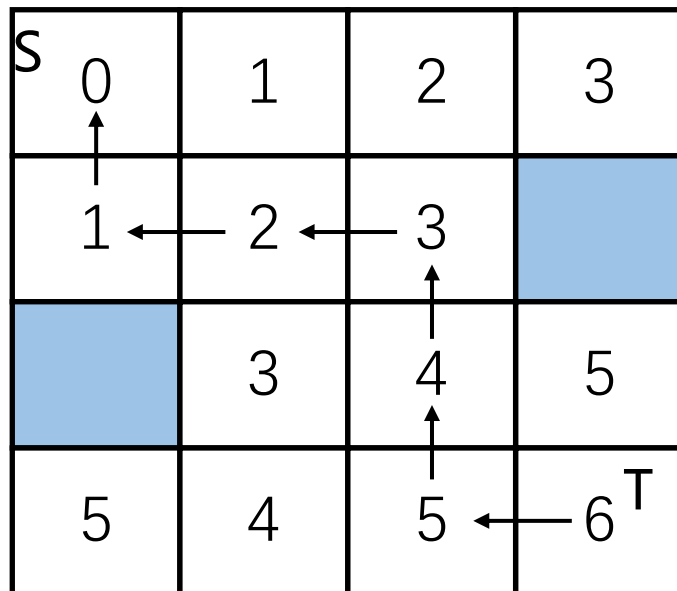


S0	1	2	3
1	2	3	
	3	4	5
5	4	5	6T

■ 通用布线算法

□ 回溯

- 从T开始回溯
- 在标记为k的顶点移动到附近任意一个标记为k-1的顶点
- 直至到达源点S



■ 通用布线算法

□ 李氏算法的时间和空间复杂度

- 对于一个 $w \times h$ 的网格结构
- 时间 = $O(wh)$
- 空间 = $O(wh \log(wh))$ (需要 $O(\log wh)$ 个比特存储每个标签)

□ 对于一个 4000×4000 的网格结构:

- 每个标签需要24比特
- 总共需要48Mbytes的内存

□ 所以, 李氏算法难以直接应用于大规模的设计当中

■ 通用布线算法

□ 李氏算法的改进

– 内存上的改进

- Akers编码方案

– 运行时间上的改进

- 起始点选择
- 双向扩展
- 边界框约束
- Hadlock算法
- Soukup算法

■ 通用布线算法

□ Akers编码方案

- 对于每个标记为 i 的顶点，相邻网格单元标签值要么是 $i-1$ ，要么是 $i+1$
- 要实现路径回溯，只需确保每个标签的前驱标签与后继标签互不相同即可
- Akers开发了一种2位编码方案
 - 用0-1标记网格单元（仅占1位存储空间）
 - 另需1位标记节点是否为障碍物

■ 通用布线算法

□ Aker编码方案

- 对于标记网格序列，应该为00110011..., 而不是010101..., 这样是为了使得每个标签的前驱和后继标签必然不同，从而保证路径回溯的正确性

S1	0	1	0
0	1	0	
	0	1	0
0	1	0	1 ^T



S	1	1	0
1	1	0	
	0	0	1
1	0	1	T

■ 通用布线算法

□ Aker编码方案

- 以下图为例，如果选择0101...的方式标记网格，从T开始回溯
- 每次回溯时选择和自身标记不同的格子

S1	0	1	0
0	1	0	
	0	1	0
0	1	0	1

T

上左均为0，选择任一方向

S	0	1	0
0	1	0	
	0	1	0
0	1	0	1

T

上左均为1，选择任一方向

S1	0	1	0
0	1	0	
	0	1	0
0	1	0	1

T

上左右均为0，此时选择右方向会导致回溯错误



■ 通用布线算法

□ Aker编码方案

- 以下图为例，如果选择00110011...的方式标记网格，从T开始回溯
- 每次回溯和前驱标记不同的格子
 - 例如前驱标记为1，那么后继只能从周围为0的格子选

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1 T

上左均为1，选择任一方向

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1 T

前驱网格是1，下一步必须
选择标记为0的网格

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1 T

前驱网格是1，下一步必须选
择标记为0的网格
此时不可能向右走

■ 通用布线算法

□ Aker编码方案

- 以下图为例，如果选择00110011...的方式标记网格，从T开始回溯
- 每次回溯和前驱标记不同的格子

两种不同回溯路径，回溯到同一个节点

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1

T

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1

T

S0	1	1	0
1	1	0	
	0	0	1
1	0	1	1

T

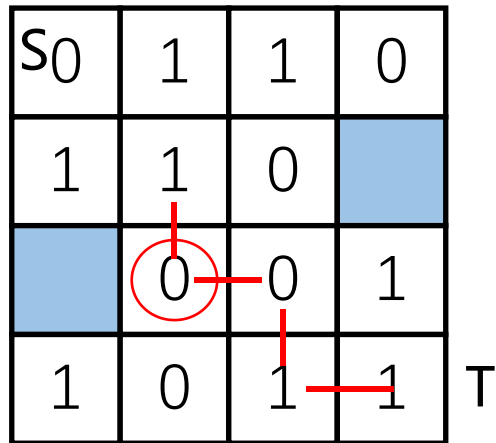
如果前面选择的不是上
而是选择向左走

前驱是1，下一步必须选择标
记为0的网格。此时只有上是
0，必须向上走

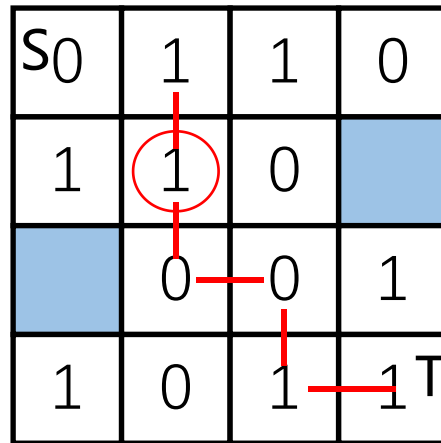
■ 通用布线算法

□ Aker编码方案

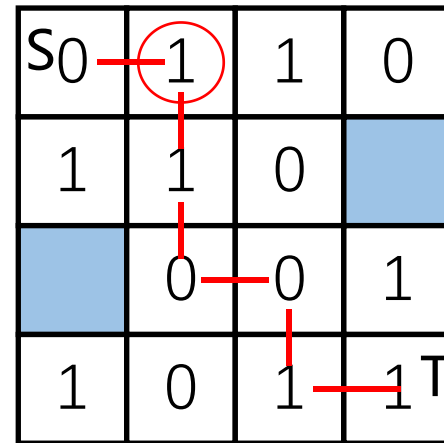
- 以下图为例，如果选择00110011...的方式标记网格，从T开始回溯
- 每次回溯和前驱标记不同的格子



前驱是0，下一步必须选择
标记为1的网格
因此不可能向下走



前驱是0，下一步必须选择
标记为1的网格



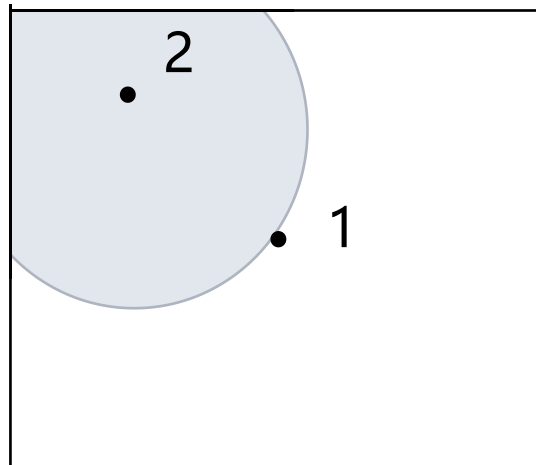
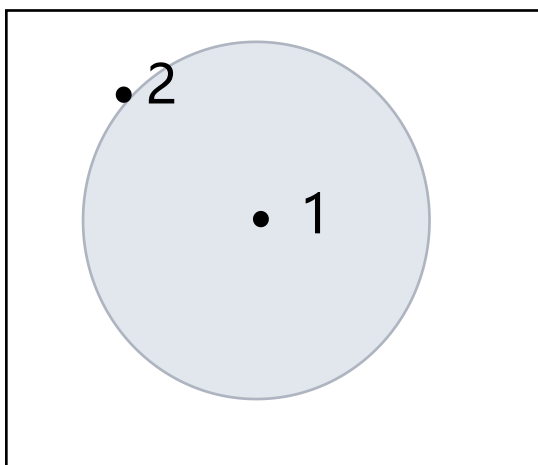
前驱是1，下一步必须选择标
记为0的网格，最终成功回溯
到起始点

■ 通用布线算法

□ 运行时间的改进

– 起始点选择

- 优先选择靠近芯片边界的点作为起点，这样在波传播的过程中可以标记更少的网格
- 如下图所示，选择1作为源节点，在波传播的过程中标记的网格比选择2作为源节点要多

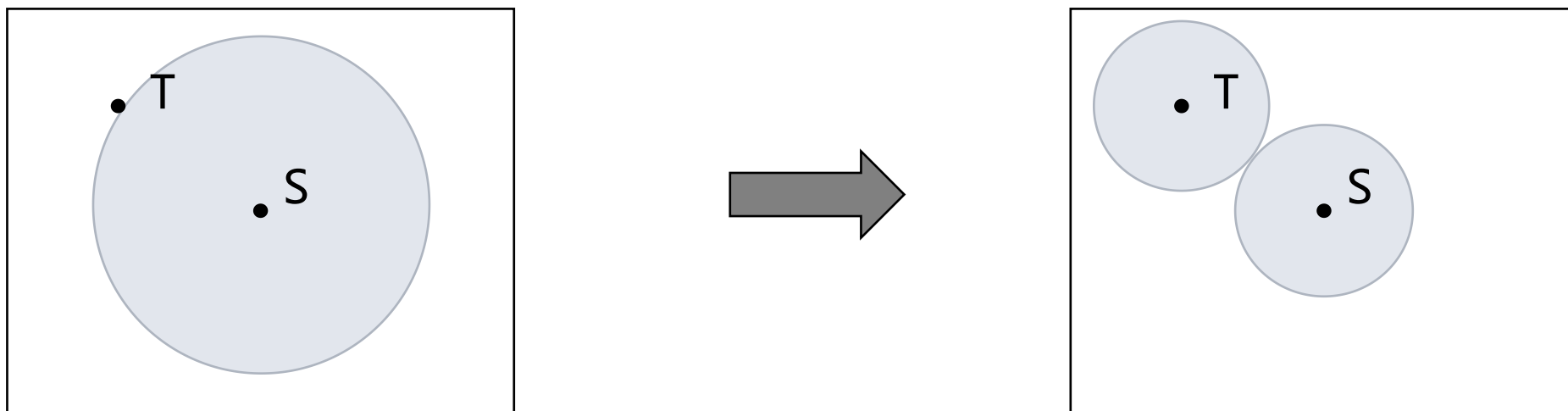


■ 通用布线算法

□ 运行时间的改进

– 双向扩展

- 同时从源节点和目标节点传播波前，缩减需要标记的区域范围
- 如下图所示，同时从源节点S和目标节点T传播波前标记的区域范围比只从源节点S传播波前标记的区域范围要小得多

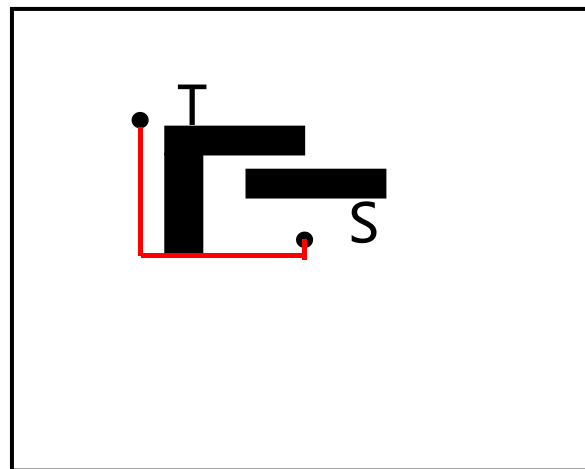
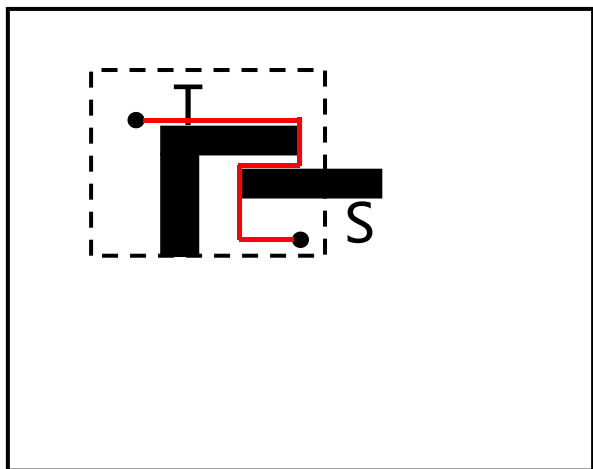


■ 通用布线算法

□ 运行时间的改进

– 边界框约束

- 在比源节点和目标节点边界框大10-20%的矩形区域内搜索。若搜索失败，则扩大矩形区域并重新执行迷宫布线。
- 需要注意的是，采用边界框方法后，李氏算法无法保证找到绝对最短路径
- 如左图所示，采用边界框方法后，找到的最短路径为左图红色线，但绝对最短路径如右图红色线所示



■ 通用布线算法

□ 运行时间的改进

– Hadlock算法

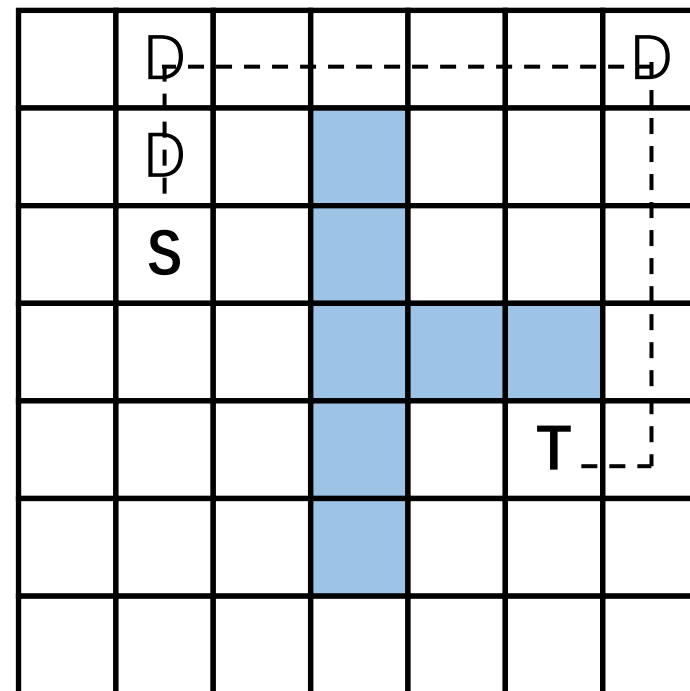
- 对于从S到T的路径P，长度为 $L(P) = MD(S, T) + 2d(P)$
- 其中， $MD(S, T)$ 为S到T的曼哈顿距离，绕行数 $d(P)$ 是**远离**T方向的网格数
- 由于S到T的曼哈顿距离是固定的，因此，最小化路径长度即为最小化绕行数

如右图所示，当沿着远离T的方向移动一格时，绕行数加一

$$d(P) = 3$$

$$MD(S, T) = 6$$

$$L(P) = 6 + 2 \times 3 = 12$$



■ 通用布线算法

□ 运行时间的改进

– Hadlock算法

- 因此，我们可以使用绕行数标记顶点
- 绕行数较小的顶点会优先被扩展
- 并且，优先选择没有绕行的路径

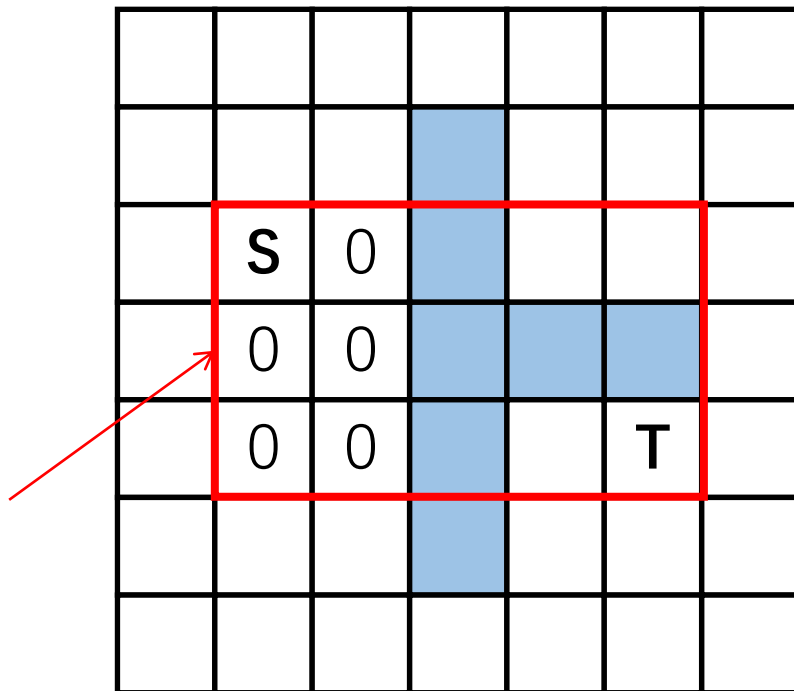
3	2	2	2	2	2	
2	1	1		2	2	
1	\$	0		2	2	
1	0	0				
1	0	0		2	T	
2	1	1		2	2	
3	2	2	-2	-2	-2	

■ 通用布线算法

□ 运行时间的改进

- Hadlock算法例子
- 根据绕行数从小到大标记网格
- 在红框区域不需要绕行
 - 如果没有障碍物，S可以直达T，距离为6
 - 在这个区域移动不会引入额外移动距离开销

这个区域内不需要绕行



■ 通用布线算法

□ 运行时间的改进

- Hadlock算法例子
- 根据绕行数从小到大标记网格
- 发现无法到达目标点
 - 在绕行数为0的周围进行拓展
 - 得到绕行数为1的区域

	1	1				
1	S	0				
1	0	0				
1	0	0			T	
	1	1				

■ 通用布线算法

□ 运行时间的改进

- Hadlock算法例子
- 根据绕行数从小到大标记网格
- 仍然无法到达目标点
 - 继续拓展绕行数为1的格子
 - 如果拓展的格子没有继续向远离T方向移动，则绕行数继承

这个区域移动没有远离T
故继承绕行数2

	2	2	2	2	2	
2	1	1		2	2	
1	S	0		2	2	
1	0	0				
1	0	0		2	T	
2	1	1		2	2	
	2	2	2	2	2	

■ 通用布线算法

□ 运行时间的改进

- Hadlock算法例子
- 根据绕行数从小到大标记网格
- 当扩展至目标节点时，根据绕行数从小到大确定路径

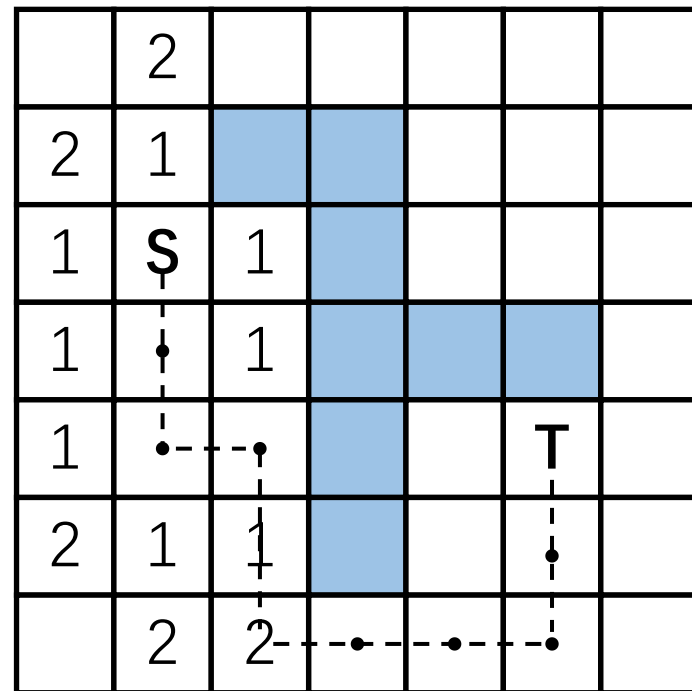
	2	2	2	2	2	
2	1	1		2	2	
1	S	0		2	2	
1	0	0				
1	0	0		2	T	
2	1	1		2	2	
	2	2	2	2	2	

■ 通用布线算法

□ 运行时间的改进

– Soukup算法

- 是一种结合DFS和BFS的方法
- 在不改变方向的情况下朝着目标方向探索（DFS）
- 如果遇到障碍物，则向相邻的节点进行广度优先搜索（BFS）
- 缺点是可能找到次优解

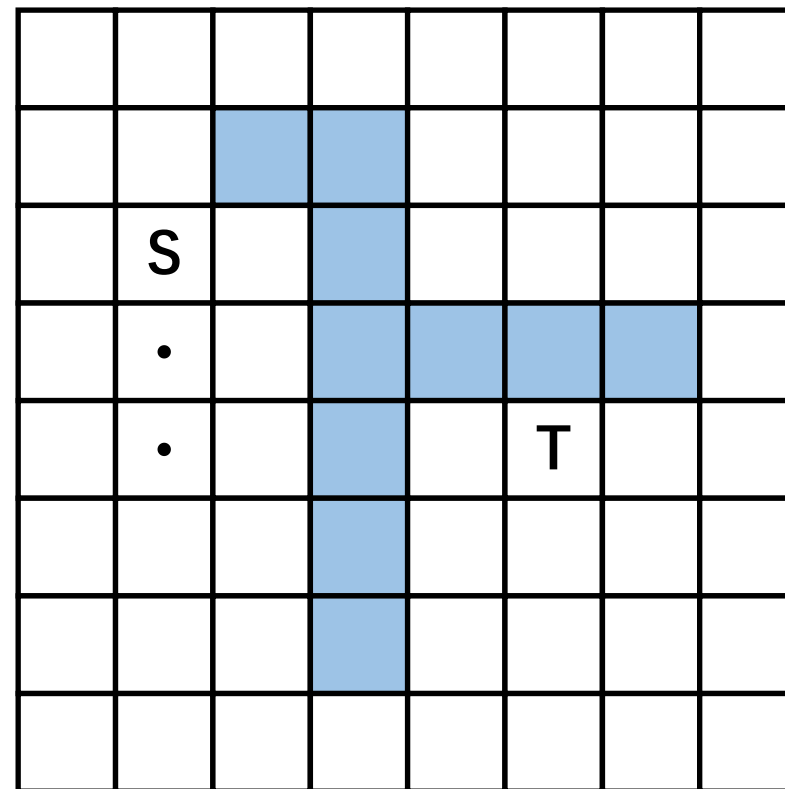


■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 首先先向下深度优先搜索，因为向下可以更接近目标节点T

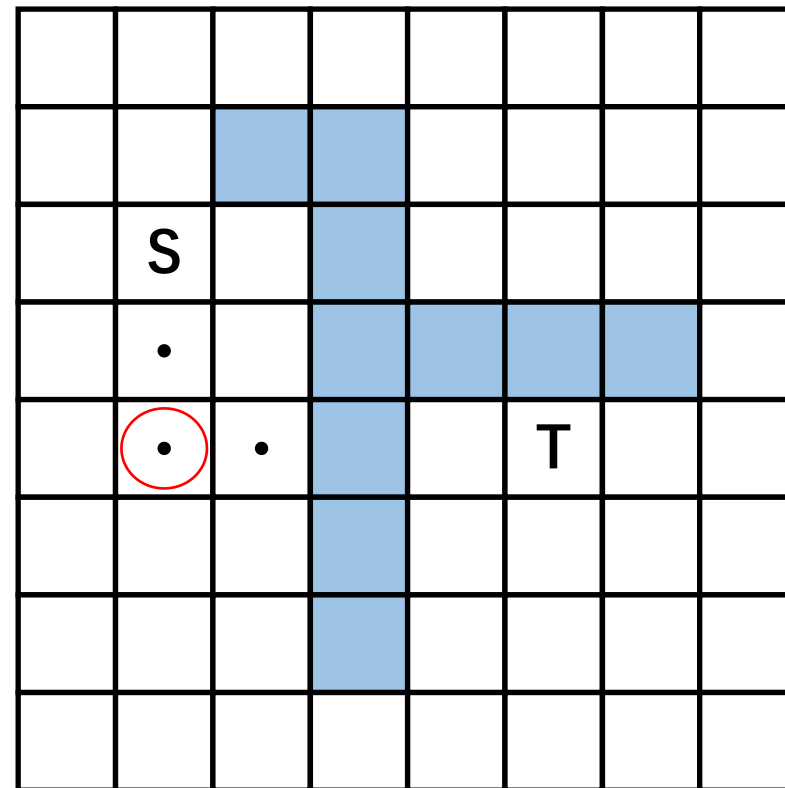


■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 由于继续向下探索会远离目标节点T，所以不选择向下探索。同时，由于向右探索更接近目标节点T，所以选择向右探索



■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 此时，继续向右探索会遇到障碍物，所以向相邻未探索的节点进行广度优先搜索

	1						
1	S	1					
1	•	1					
1	•	•		T			
	1	1					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 未发现节点向某个方向探索可以更加接近目标节点T，因此继续广度优先搜索

	2						
2	1						
1	S	1					
1	•	1					
1	•	•			T		
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 经过两次广度优先探索后，发现在标记节点向右探索可以更接近目标节点T，所以向右探索

	2	•	•	•	•		
2	1						
1	S	1					
1	•	1					
1	•	•		T			
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

➤ 此时，向下探索会更接近目标节点T，因此向下搜索

	2	•	•	•	•		
2	1				•		
1	S	1			•		
1	•	1					
1	•	•			T		
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 继续向下搜索会遇到障碍物，因此向相邻未探索的节点进行广度优先搜索（优先扩展上一次深度优先搜索的节点）

	2	•	•	•	•	1	
2	1			1	•	1	
1	S	1		1	•	1	
1	•	1					
1	•	•			T		
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 未发现节点向某个方向探索可以更加接近目标节点T，因此继续广度优先搜索

	2	•	•	•	•	1	2
2	1			1	•	1	2
1	S	1		1	•	1	2
1	•	1					
1	•	•			T		
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 经过两次广度优先探索后，发现在标记节点向下探索可以更接近目标节点T，所以向下探索
- 最后继续向左进行深度优先搜索即可到达目标节点T

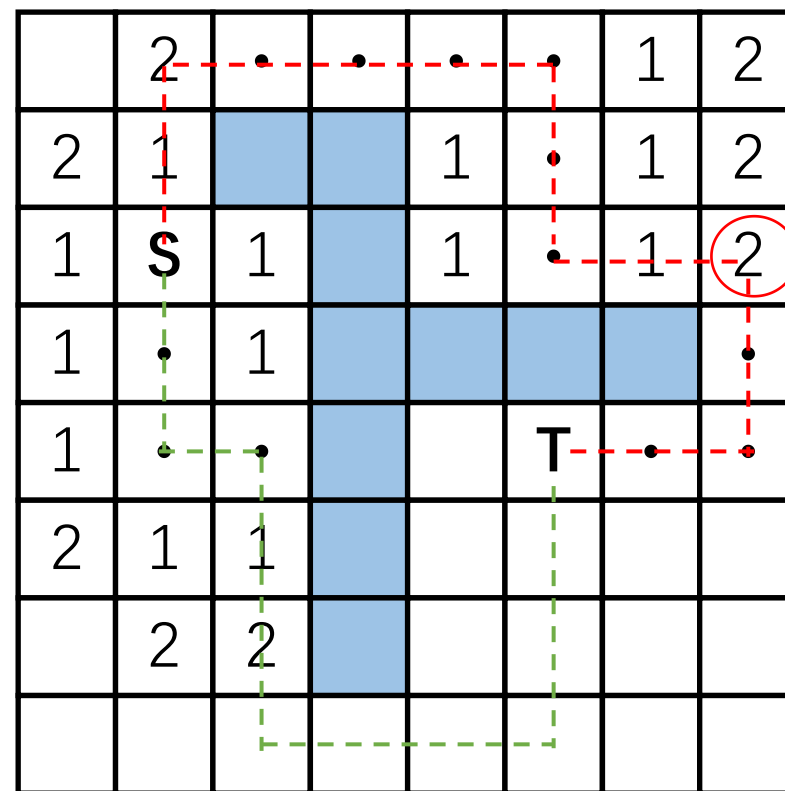
	2	•	•	•	•	1	2
2	1			1	•	1	2
1	S	1		1	•	1	2
1	•	1					•
1	•	•			T	•	•
2	1	1					
	2	2					

■ 通用布线算法

□ 运行时间的改进

– Soukup算法例子

- 最终得到的路径如右图红线所示，长度为14，但是这并不是最优解
- 最优解路径如右图绿线所示，长度为12



■ 通用布线算法

□ 迷宫布线算法的主要缺陷是内存占用高和运行时间长

□ 线搜索算法

- 通过用线段表示布线空间和路径来缓解这个问题，但是会牺牲解的质量

- 输入

 - 一个平面矩形图

 - 图上的两个点S和T

 - 障碍物建模为阻塞的矩形

- 目标

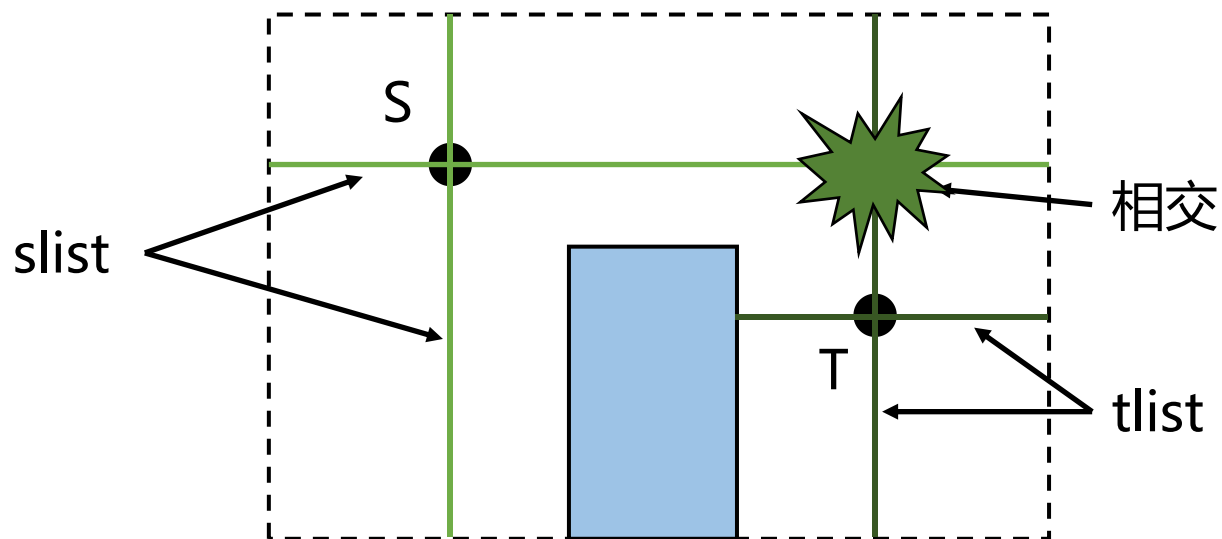
 - 找到连接S和T的路径

■ 通用布线算法

□ 线搜索算法

– Mikami-Tabuchi算法

- 维护两个线段列表，分别为起始点列表（slist）和终点列表（tlist）
- 如果来自 slist 的线段与来自 tlist 的线段相交，则找到一条路径
- 否则，从逃逸点生成新的线段

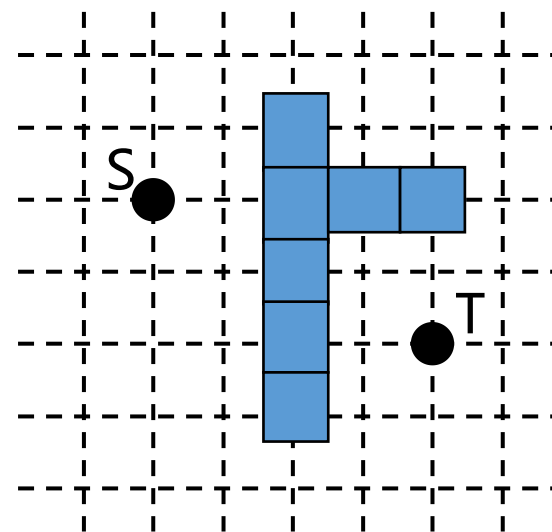


■ 通用布线算法

□ 线搜索算法

– Mikami-Tabuchi算法例子

- 给定如下平面图
- 起始节点为S，目标节点为T
- 障碍物建模为蓝色矩形

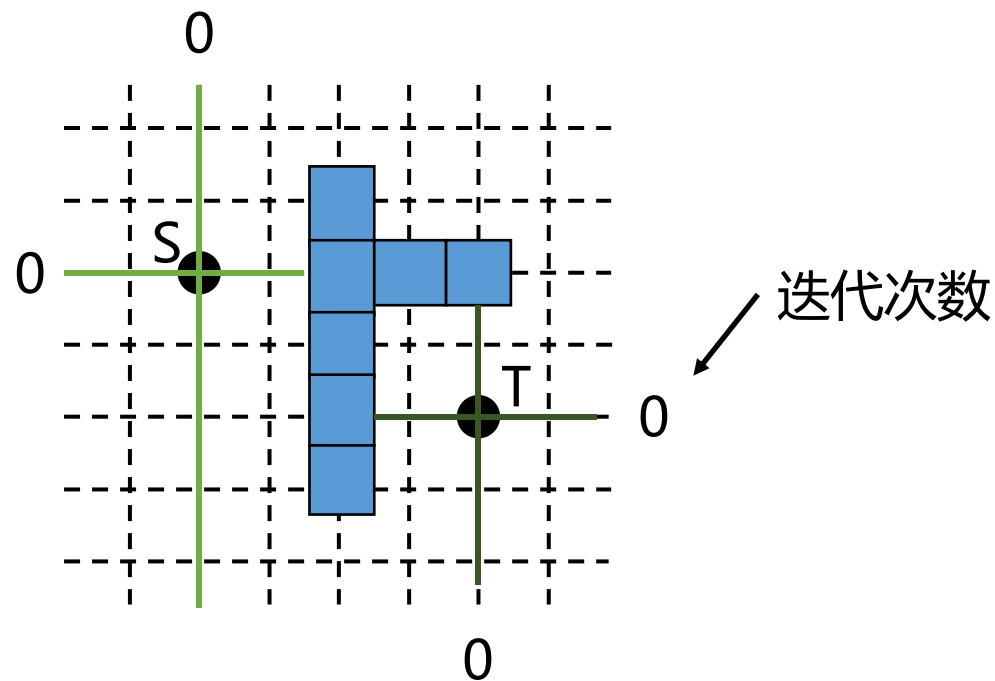


■ 通用布线算法

□ 线搜索算法

– Mikami-Tabuchi算法例子

- 首先将S和T作为基础点，然后生成四条（两条水平、两条垂直）穿过这些基础点的0级线段，这些线段延伸至边界或者障碍物为止



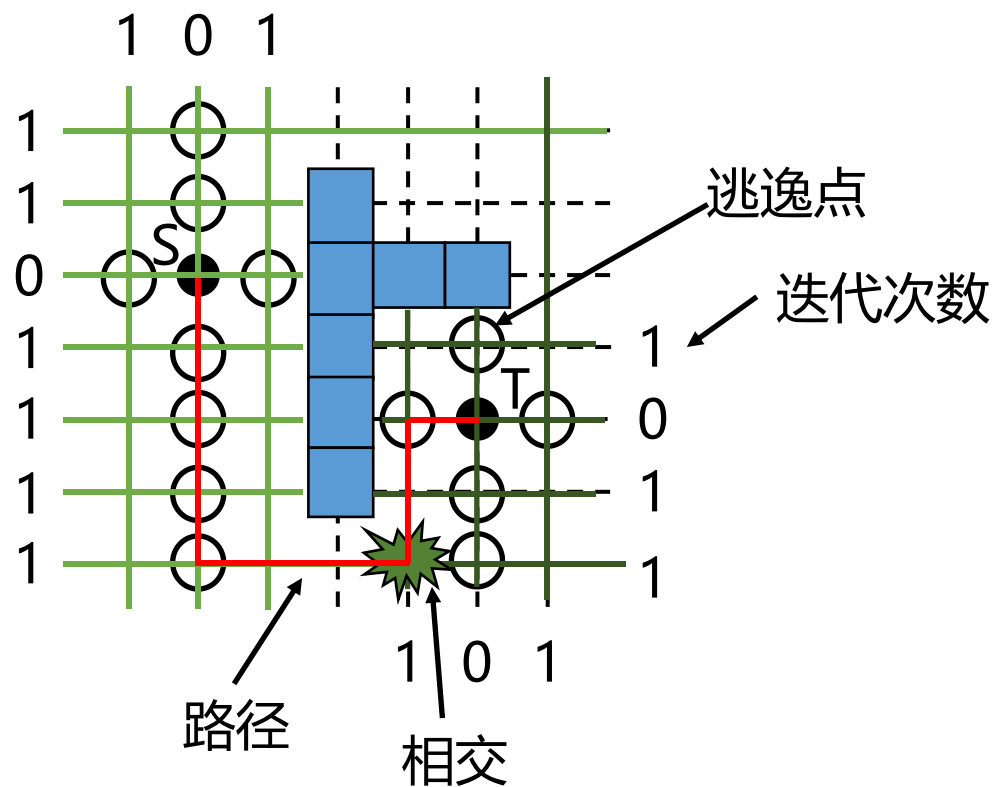
浅绿色线段是从S生成的线段，深绿色的线段是从T生成的线段

■ 通用布线算法

□ 线搜索算法

– Mikami-Tabuchi算法例子

- 接着，将第i级线段上的**每个格点**迭代设置为逃逸点，并生成与之垂直的第i+1级线段。此过程重复进行，直到从S生成的线段与从T生成的线段相交，此时可通过从该交点回溯至S和T来建立路径。



浅绿色线段是从S生成的线段，深绿色的线段是从T生成的线段

■ 通用布线算法

□ 线搜索算法

– Hightower算法

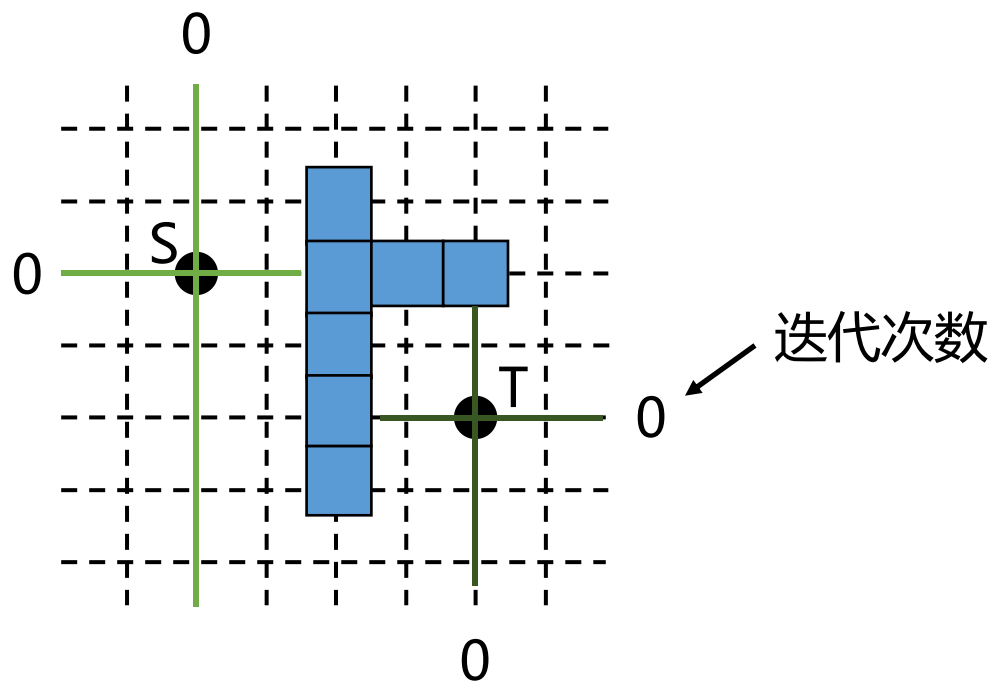
- 原理与Mikami-Tabuchi算法类似，区别在于仅从每条线段中选取一个逃逸点
- 相比于Mikami-Tabuchi算法，Hightower算法处理的线段更少，更加节省内存
- 但是Hightower算法存在一个缺点：即使存在路径，Hightower算法也可能会找不到，因为能否找到路径取决于选取的逃逸点

■ 通用布线算法

□ 线搜索算法

– Hightower算法例子

- 给定如下平面图
- 首先将S和T作为基础点，然后生成四条（两条水平、两条垂直）穿过这些基础点的0级线段，**这些线段延伸至边界或者障碍物为止**



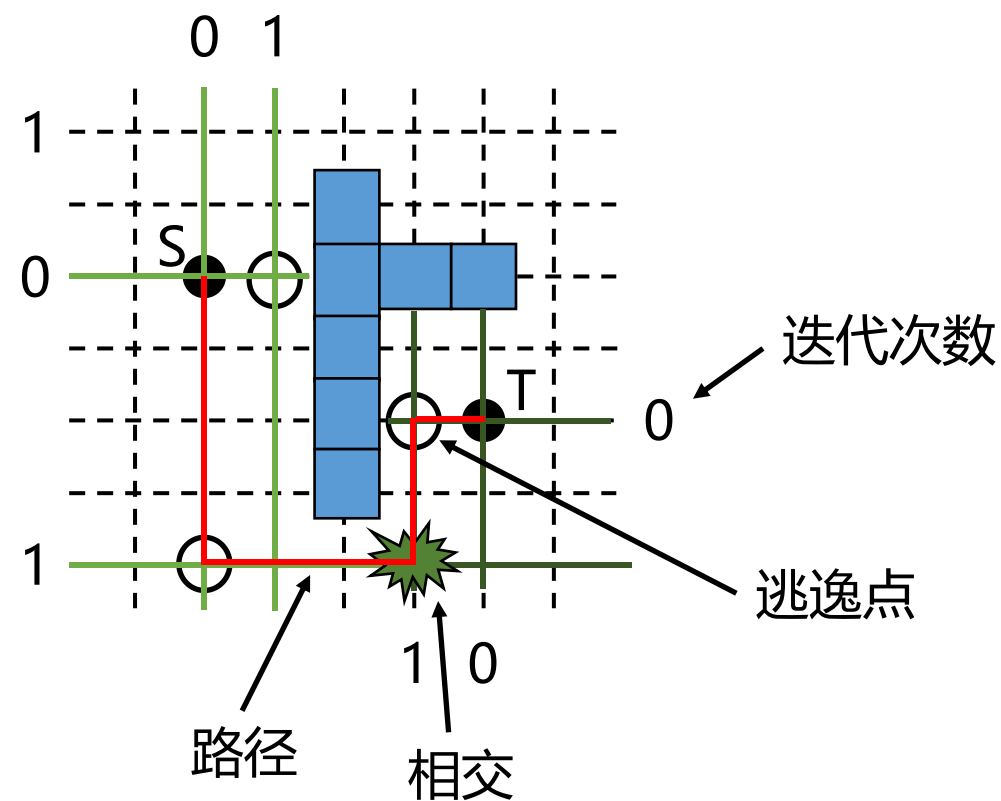
浅绿色线段是从S生成的线段，深绿色的线段是从T生成的线段

■ 通用布线算法

□ 线搜索算法

– Hightower算法例子

- 给定如下平面图
- 首先将S和T作为基础点，然后生成四条（两条水平、两条垂直）穿过这些基础点的0级线段，**这些线段延伸至边界或者障碍物为止**
- 接着，在第i级线段上选择一个格点迭代设置为逃逸点，并生成与之垂直的第i+1级线段。此过程重复进行，直到从S生成的线段与从T生成的线段相交，此时可通过从该交点回溯至S和T来建立路径。



浅绿色线段是从S生成的线段，深绿色的线段是从T生成的线段

■ 通用布线算法

□ 各个方法对比

	迷宫布线			线搜索	
	Lee	Soukup	Hadlock	Mikami	Hightower
时间	$O(MN)$	$O(MN)$	$O(MN)$	$O(L)$	$O(L)$
空间	$O(MN)$	$O(MN)$	$O(MN)$	$O(L)$	$O(L)$
如果存在 路径能否 找到?	√	√	√	√	×
路径一定 是最短的 吗?	√	×	√	×	×

其中，M和N分别对应网格的长和宽，L对应生成线段的数量

感谢！
